Laporan Evaluasi Tengah Semester

Author(s):	Nasrul (NRP. 5022241067)
Teacher:	Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
Class name:	Sistem Robot Otonom
GitHub link:	`https://github.com/nasruul/ETS-SRO-Nasrul.git`
YouTube video link (Demo Proses):	`https://youtu.be/beXEb19Ait4`
YouTube video link (Demo Hasil 3D):	`https://youtu.be/66v9deKGp1Y`

Ringkasan Proyek

Laporan Evaluasi Tengah Semester (ETS) ini mendokumentasikan perancangan dan implementasi sistem pemetaan 3D (*3D Mapping*) secara *real-time* untuk wahana *quadcopter* pada simulator CoppeliaSim. Sistem ini mengintegrasikan sensor *fast 3D Laser Scanner* dengan skrip Python eksternal melalui protokol antarmuka *ZeroMQ Remote API*. Pengendalian wahana menggunakan pengontrol GUI berbasis Tkinter yang dilengkapi mekanisme *multi-key holding* untuk pergerakan halus, serta algoritma pemrosesan *point cloud* untuk rekonstruksi peta 3D lingkungan sekitar secara presisi.

Sistem Kontroler dan Algoritma Pergerakan

1. Pengontrol GUI dan Multi-Key Listener

Pengontrol penerbangan dibangun menggunakan pustaka Tkinter dengan menerapkan antarmuka pemantau papan ketik simultan (*multi-key listener*). Metode ini menyelesaikan keterbatasan antarmuka standar yang hanya mampu membaca satu input tombol dalam satu waktu.

- **Penanganan Event Keyboard:** Setiap kali tombol diketik (`KeyPress`), karakter dimasukkan ke dalam himpunan data (*set*) `active_keys`. Saat tombol dilepas (`KeyRelease`), karakter dihapus dari himpunan tersebut.
- **Threading Berkecepatan Tinggi:** Fungsi `update_loop()` dijalankan pada *background thread* terpisah dengan laju pembaruan 50 Hz (`sleep(0.02s)`). Vektor pergerakan dihitung berdasarkan kombinasi tombol aktif:

$$\begin{aligned} \Delta x &= (\mathbf{1}_d - \mathbf{1}_a) \cdot v \\ \Delta y &= (\mathbf{1}_w - \mathbf{1}_s) \cdot v \\ \Delta z &= (\mathbf{1}_i - \mathbf{1}_k) \cdot v \end{aligned} \tag{1}$$

  di mana $v = 0.08$ m/step merupakan konstanta kecepatan pergerakan, dan $\mathbf{1}_{key}$ bernilai 1 jika tombol terdeteksi aktif di dalam himpunan.

Transformasi Koordinat dan Pemrosesan Point Cloud

1. Transformasi Homogen 3D

Data mentah yang ditangkap oleh sensor *3D Laser Scanner* dinyatakan dalam sistem koordinat lokal sensor $P_{\text{local}} = [x_l, y_l, z_l]^T$. Untuk memetakan titik ke dalam koordinat dunia (*World Frame*) P_{global}, diterapkan matriks transformasi homogen $T_{\text{sensor}}^{\text{world}} \in \mathbb{R}^{4\times4}$:

$$\begin{bmatrix} x_g \\ y_g \\ z_g \\ 1 \end{bmatrix} = T_{\text{sensor}}^{\text{world}} \cdot \begin{bmatrix} x_l \\ y_l \\ z_l \\ 1 \end{bmatrix} \tag{2}$$

Matriks $T_{\text{sensor}}^{\text{world}}$ diekstrak secara berkala dari CoppeliaSim melalui fungsi `sim.getObjectMatrix()` guna menyesuaikan perubahan orientasi dan posisi drone saat terbang.

2. Penurasan Voxel Grid dan Visualisasi

Untuk mengurangi beban memori komputasi dari penumpukan data titik yang identik, diterapkan algoritma penurasan *voxel grid* ($s = 0.03$ m):

$$V_{(x,y,z)} = \left\lfloor \frac{P_{\text{global}}}{s} \right\rfloor \tag{3}$$

Hanya satu koordinat representatif yang diambil dari setiap kubus *voxel* menggunakan fungsi indeks unik `numpy.unique()`. Selanjutnya, nilai elevasi Z dinormalisasi ke rentang $[0, 1]$ dan dipetakan ke skema warna `turbo` dengan batang indikator skala (*colorbar*) untuk memberikan gambaran gradasi ketinggian objek secara visual.

Sistem Kontroler Drone dan Antarmuka GUI

1. Arsitektur Antarmuka Interaktif (Tkinter)

Untuk mengendalikan wahana *quadcopter* secara presisi dan intuitif selama proses pemetaan 3D, dikembangkan antarmuka berbasis *Graphical User Interface* (GUI) menggunakan pustaka Tkinter. Antarmuka ini menyediakan tombol-tombol navigasi visual (Maju, Mundur, Kiri, Kanan, Naik, Turun) serta tombol eksekusi **"Stop & Plot 3D Map"** untuk menghentikan pemindaian dan memicu pemrosesan grafik.

2. Mekanisme Multi-Key Listener dan Event Handling

Pustaka GUI standar umumnya hanya mampu memproses satu masukan *keyboard* secara bergantian. Untuk menghasilkan pergerakan wahana yang halus, natural, dan mendukung gerakan komposit (diagonal), diimplementasikan algoritma *Multi-Key Listener* berbasis *Set Data Structure*:

- **Penanganan Event `KeyPress` & `KeyRelease`:** Saat sebuah tombol dialfanumerik ditekan, event `<KeyPress>` memasukkan karakter tombol ke dalam himpunan (*set*) `active_keys`. Ketika tombol dilepas, event `<KeyRelease>` menghapus karakter tersebut dari himpunan.
- **Kombinasi Tombol Simultan:** Melalui pemantauan himpunan `active_keys`, kontroler dapat mengidentifikasi penekanan beberapa tombol sekaligus. Sebagai contoh, penekanan tombol `W` dan `D` secara bersamaan akan menghasilkan kombinasi vektor pergerakan maju-kanan secara konkrit.

3. Threading dan Penentuan Vektor Pergerakan

Proses kalkulasi posisi target drone dipisahkan dari *Main Thread* GUI menggunakan *Daemon Thread* terpisah. Hal ini bertujuan untuk mencegah pembekuan (*freezing*) antarmuka saat pertukaran data dengan CoppeliaSim berlangsung.

Fungsi `update_loop()` mengeksekusi perhitungan posisi target pada frekuensi 50 Hz ($\Delta t = 0.02$ s). Vektor translasi $(\Delta x, \Delta y, \Delta z)$ dihitung secara dinamis dari himpunan tombol aktif melalui persamaan:

$$\begin{aligned} \Delta x &= (\mathbf{1}_{\{d \in \text{keys}\}} - \mathbf{1}_{\{a \in \text{keys}\}}) \cdot v \\ \Delta y &= (\mathbf{1}_{\{w \in \text{keys}\}} - \mathbf{1}_{\{s \in \text{keys}\}}) \cdot v \\ \Delta z &= (\mathbf{1}_{\{i \in \text{keys}\}} - \mathbf{1}_{\{k \in \text{keys}\}}) \cdot v \end{aligned} \tag{4}$$

di mana $v = 0.08$ m/step mewakili konstanta kecepatan translasi, dan $\mathbf{1}_{\{\cdot\}}$ adalah fungsi indikator penekanan tombol. Vektor posisi baru $P_{\text{cmd}} = P_{\text{lama}} + [\Delta x, \Delta y, \Delta z]^T$ kemudian dikirimkan ke objek target *quadcopter* di CoppeliaSim via fungsi `sim.setObjectPosition()`.

Table 1: Pemetaan Tombol Kontroler dan Aksi Pergerakan Drone

Tombol Kontrol	Aksi Pergerakan	Sumbu Koordinat Target
`W`	Maju	$+Y$ (Planar)
`S`	Mundur	$-Y$ (Planar)
`A`	Kiri	$-X$ (Planar)
`D`	Kanan	$+X$ (Planar)
`I`	Naik	$+Z$ (Elevasi Vertikal)
`K`	Turun	$-Z$ (Elevasi Vertikal)



Figure 1: Tampilan Antarmuka GUI Kontroler Penerbangan Drone Berbasis Tkinter.

Hasil Simulasi dan Pembahasan

Pengujian dilakukan dengan mengarahkan drone mengelilingi area simulasi. Hasil rekonstruksi pemetaan 3D *point cloud* beserta trajektori lintasan penerbangan diperlihatkan pada Gambar 2, sedangkan ringkasan parameter pengujian dirangkum pada Tabel 2.

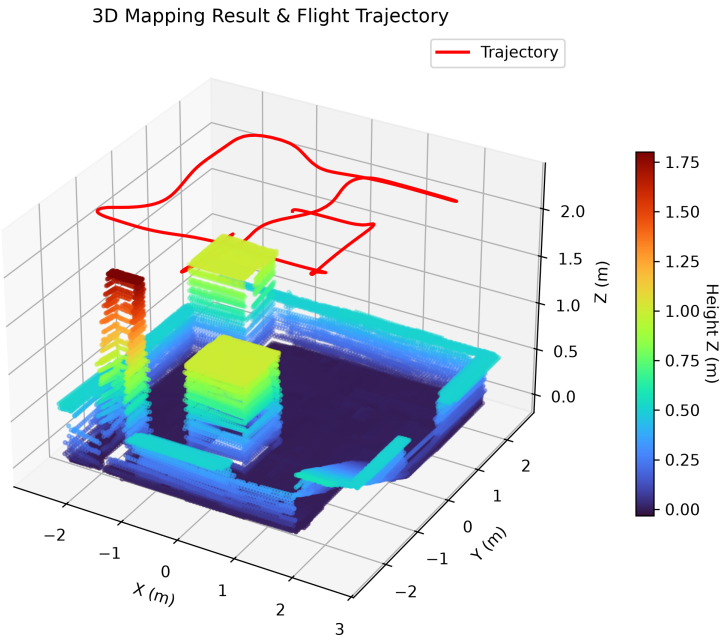


Figure 2: Hasil Rekonstruksi 3D Point Cloud dan Trajektori Penerbangan Drone.

Table 2: Ringkasan Parameter Pengujian dan Performa Simulasi

No	Parameter	Nilai / Keterangan
1	Ukuran Voxel Filter	0.03 m (3 cm)
2	Sinyal Buffer Sensor	`fast3D_data`
3	Frekuensi Kontroler	50 Hz ($\Delta t = 0.02$ s)
4	Pemetaan Warna	Colormap `turbo` dengan *Colorbar Z*

Kesimpulan

Sistem pemetaan 3D menggunakan *quadcopter* pada simulator CoppeliaSim berbasis Python ZMQ Remote API telah berhasil diimplementasikan. Kombinasi antarmuka *multi-key holding* berbasis Tkinter dan algoritma transformasi koordinat homogen menghasilkan proses akuisisi data *point cloud* yang responsif dan presisi. Penurasan *voxel grid* terbukti efektif mereduksi redundansi data sehingga peta 3D dan trajektori penerbangan dapat divisualisasikan dengan jelas.